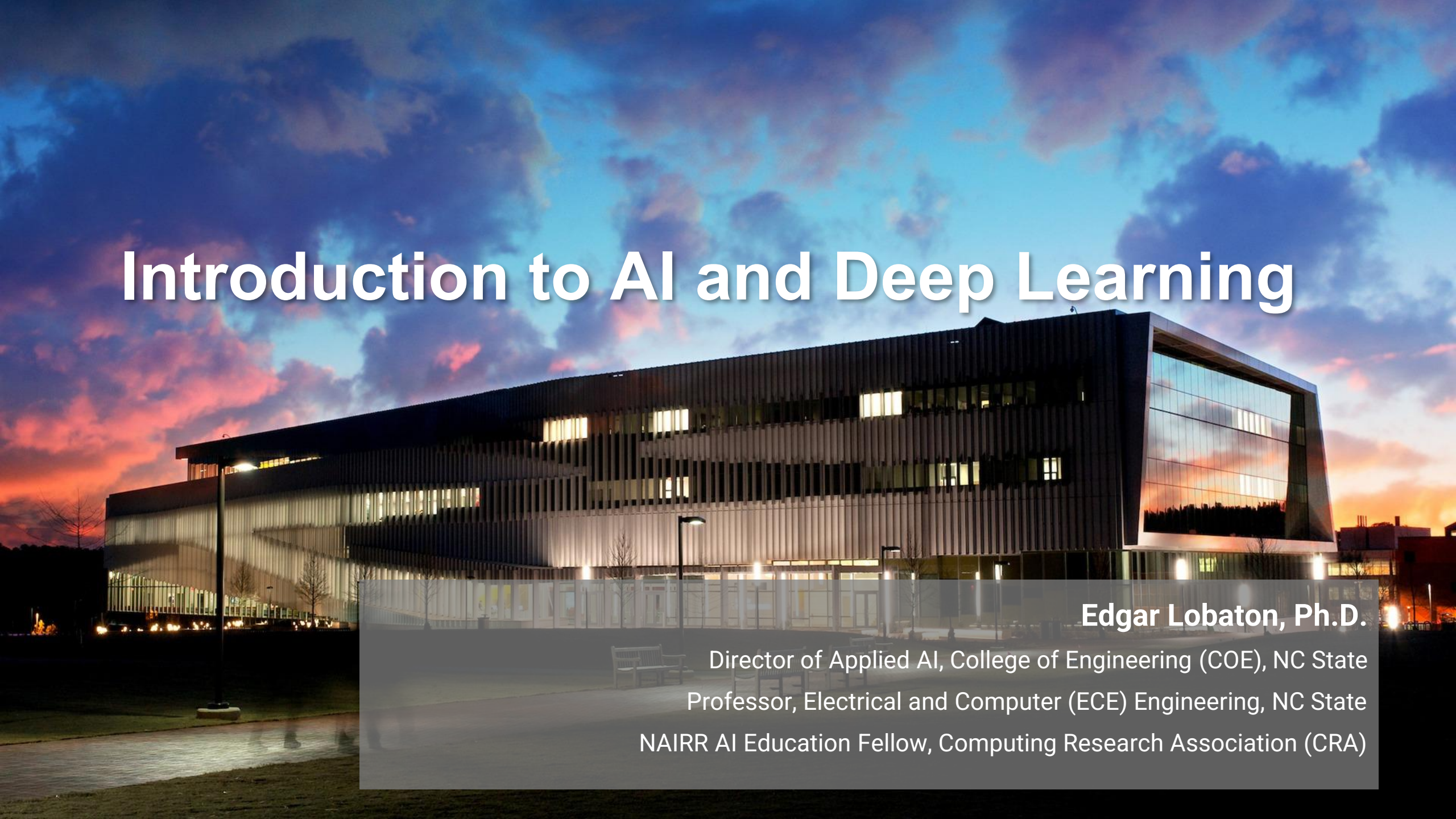


# Introduction to AI and Deep Learning

A photograph of a modern, multi-story building with a large glass facade, illuminated from within and reflecting the vibrant colors of a sunset sky. The sky is filled with clouds in shades of blue, orange, and pink. The building's architecture features a mix of dark and light panels, and the interior lights are visible through the windows. The foreground shows a paved area and some landscaping.

**Edgar Lobaton, Ph.D.**

Director of Applied AI, College of Engineering (COE), NC State  
Professor, Electrical and Computer (ECE) Engineering, NC State  
NAIRR AI Education Fellow, Computing Research Association (CRA)

# Course Overview

---



<https://github.com/lobaton/DM2026-Deep-Learning-with-Python>



# What are AI and Machine Learning?

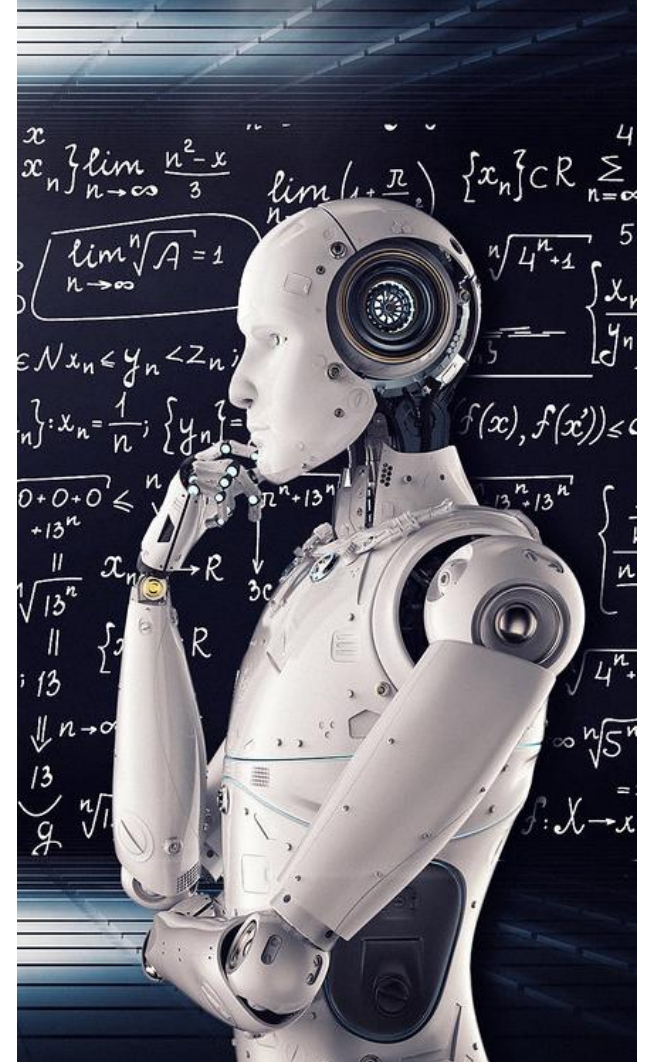
**Edgar Lobaton, Ph.D.**

Director of Applied AI, College of Engineering (COE), NC State  
Professor, Electrical and Computer (ECE) Engineering, NC State  
NAIRR AI Education Fellow, Computing Research Association (CRA)

# What is AI?

---

- AI allows computers to perform tasks that typically require human intelligence. AI systems can "learn" from information, recognize patterns, predict, make decisions and act on them.
- Examples:** Generative AI (ChatGPT, Meshy AI), recommendation (Netflix, Amazon), and code assistants (Codex, Claude Code).
- A course on AI studies and designs intelligent agents. These agents can perceive their environment, reason about information, and take autonomous actions to achieve a specific goal.

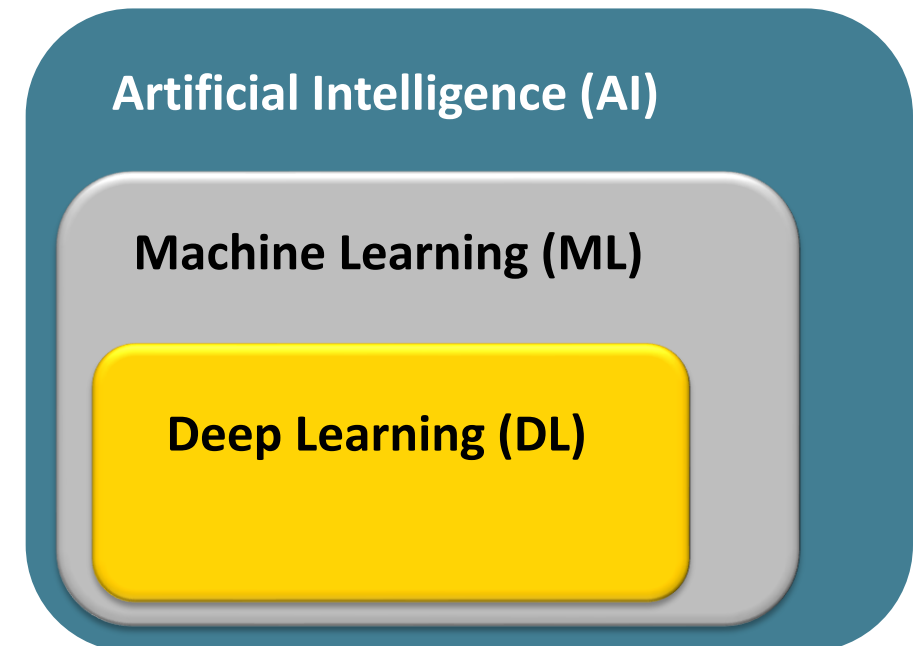




# What is ML?

---

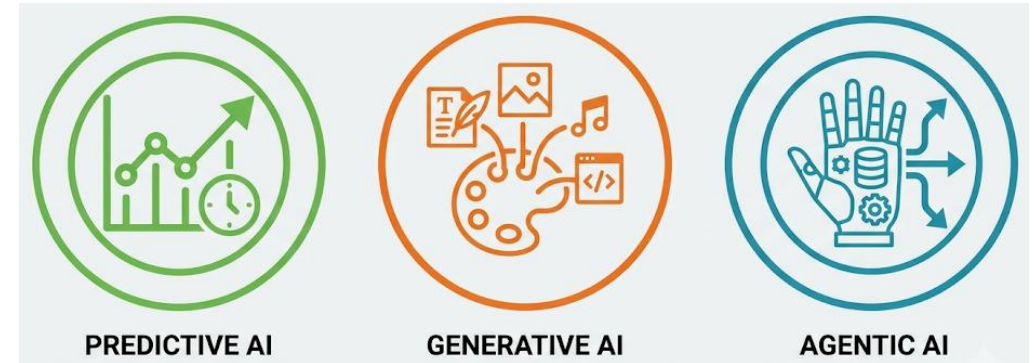
- **Machine Learning (ML)** is a subset of AI that focuses on algorithms that improve through experience. Instead of a human writing specific rules for every scenario, the programmer writes algorithms that allows the computer to find patterns in data.
- **Classical ML** requires feature engineering (i.e., humans tell the computer what parts of the data are important).
- **Deep Learning (DL)** is a subset of ML that discovers the features. Neural Networks are the foundation of DL.



# Types of AI

---

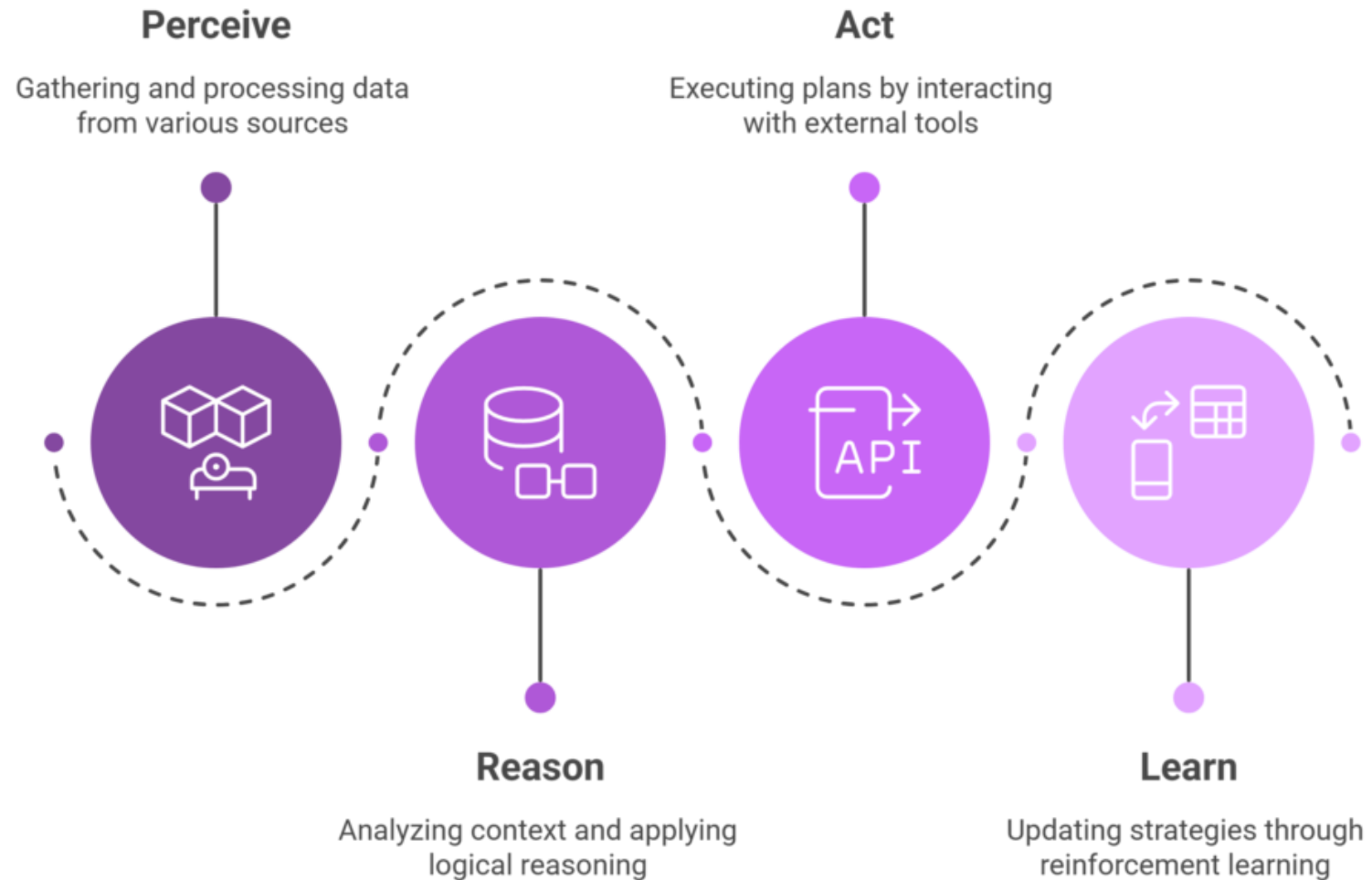
- **Predictive AI: What will happen?**
  - Produces a forecast, score or classification
  - Uses pattern recognition on historical data
  - Examples: Predictive maintenance / weather forecast
- **Generative AI: What can be created?**
  - Produces new content (text, images, code)
  - Uses sampling from a learned distribution
  - Examples: Generate a design / draft a report



- **Agentic AI: What should be done?**
  - Completes multi-step tasks
  - Uses reasoning and tools through multiple iterations often taking advantage of generative models.
  - Examples: An agent that runs a simulation, checks results and revises without a human in the loop.

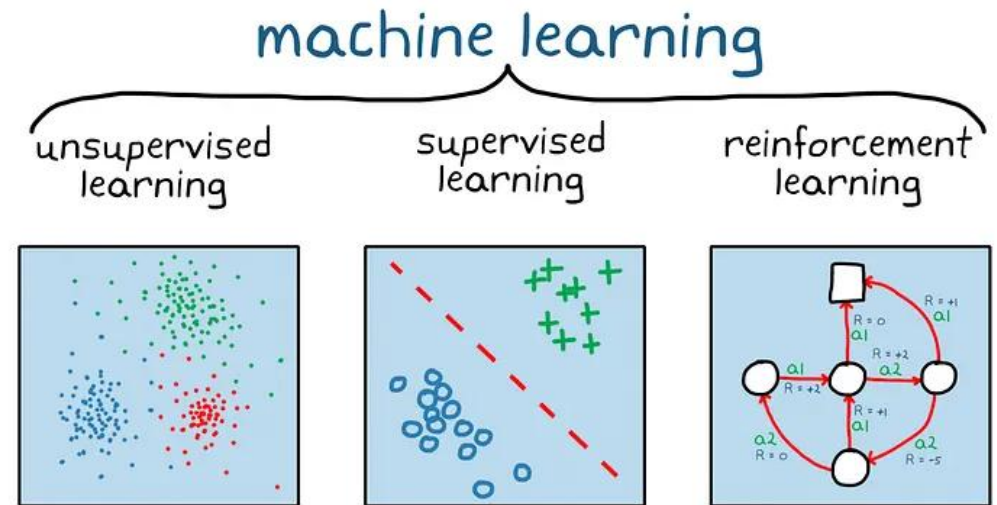
# Agentic AI Pipeline

---



# Types of ML

- **Supervised Learning:**  
What is the output for this input?
  - Uses labeled images
  - Learns a mapping from input to output
  - Example: Crack detection for labeled images
- **Unsupervised Learning:**  
What pattern exists in this data?
  - Uses unlabeled examples
  - Finds groups, structure or distributions
  - Example: Clustering pump failures signatures from sensor logs



- **Reinforcement Learning:**  
What action maximizes reward over time?
  - Learns through trial-and-error
  - Takes actions, receives rewards, adjusts behavior
  - Example: A robotic arm learning to grasp an object



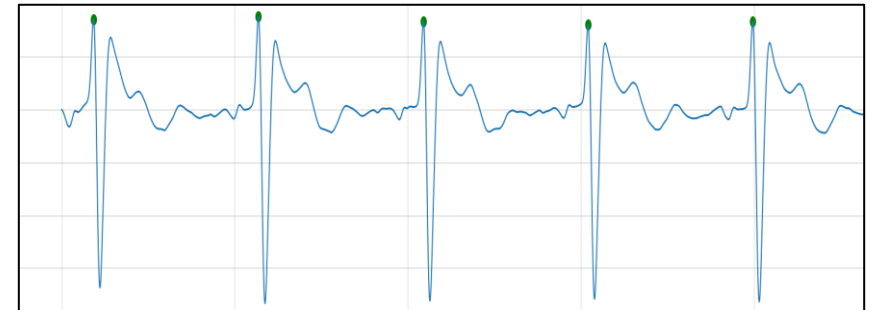
# Supervised Learning

Given that data is provided in input/output pairs, then ML can help identify the mapping between them. That is,  $\text{output} = f(\text{input})$ .



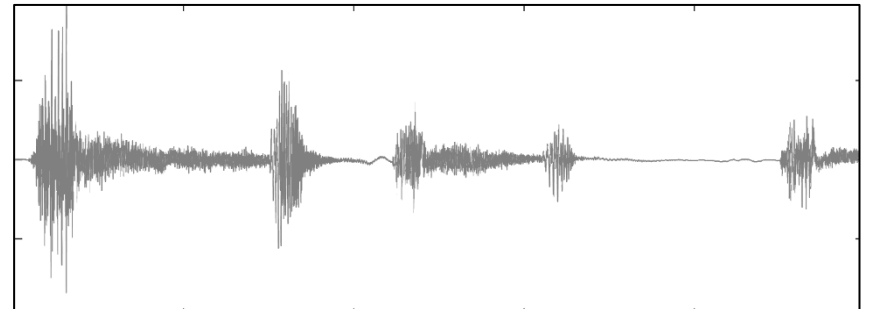
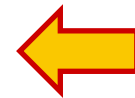
→ Cat

90 bpm



→ Dog

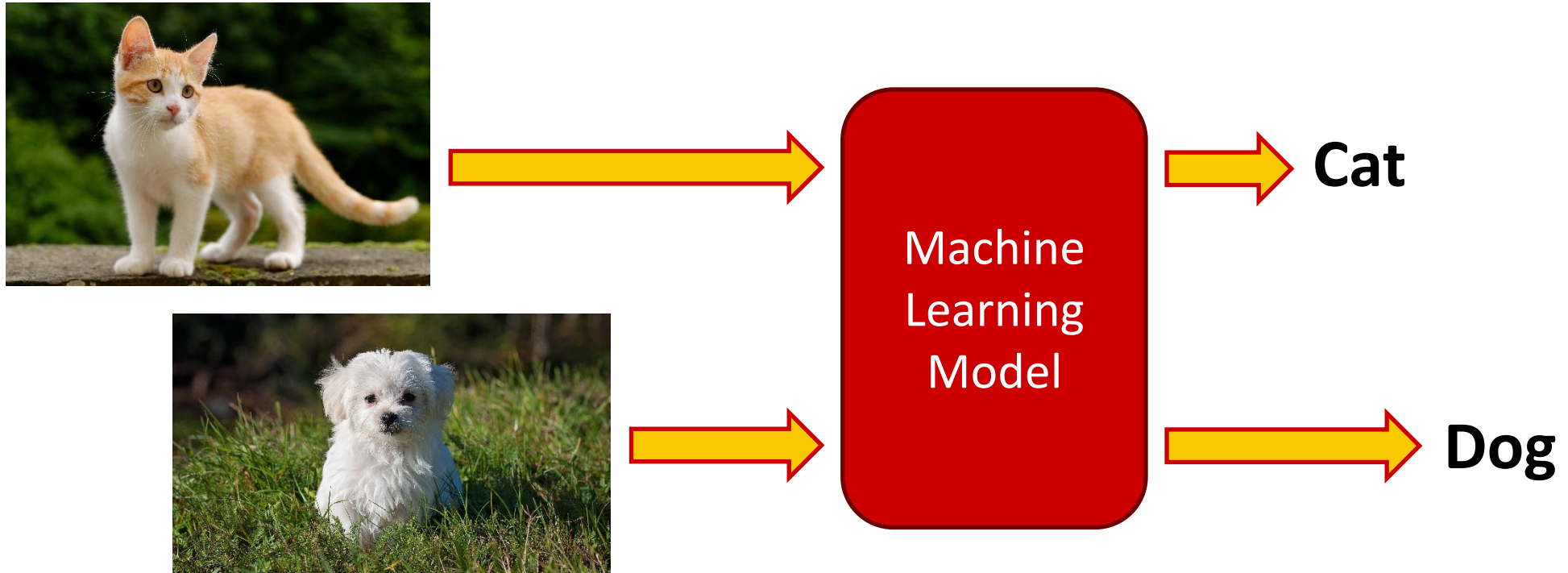
Stress



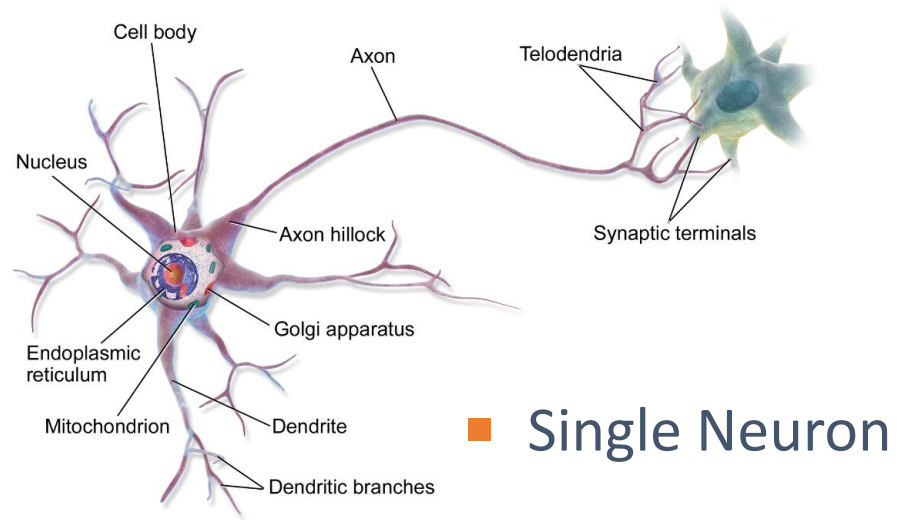
# Supervised Learning

---

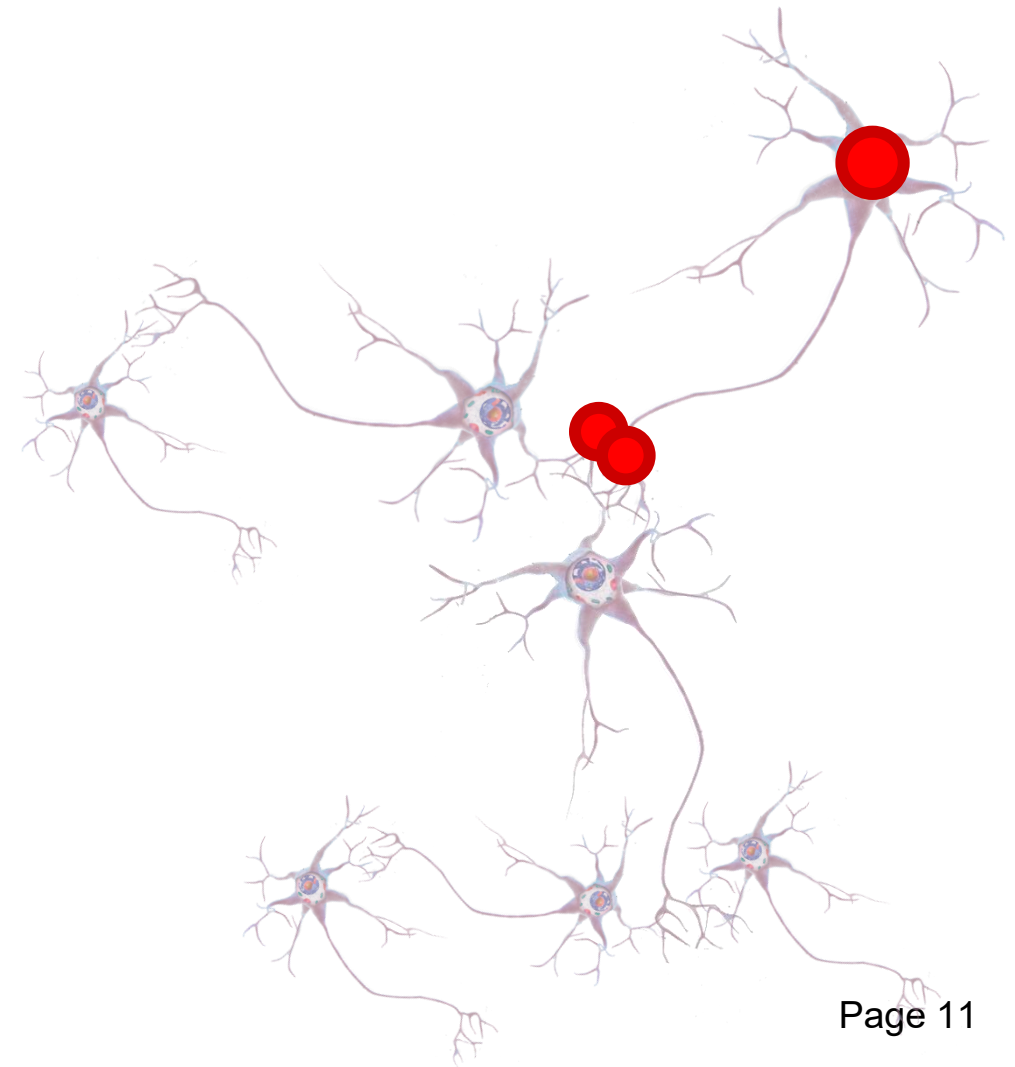
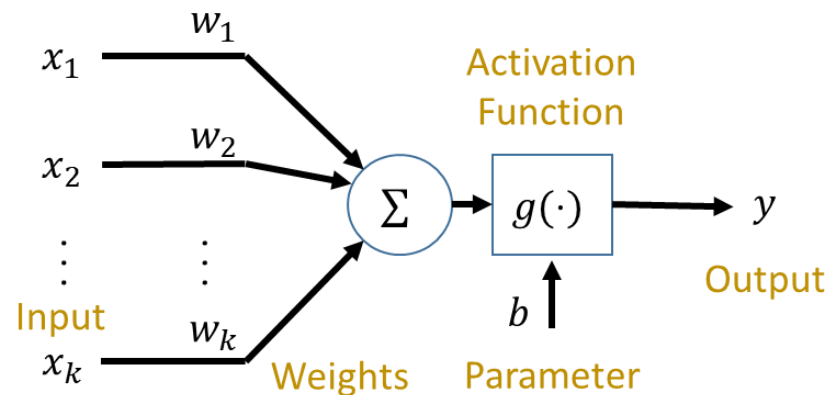
Think of a supervised learning model as a function or a black box (at least for now) that maps inputs (images) to outputs (labels in this case)



# Artificial Neural Network (NN)

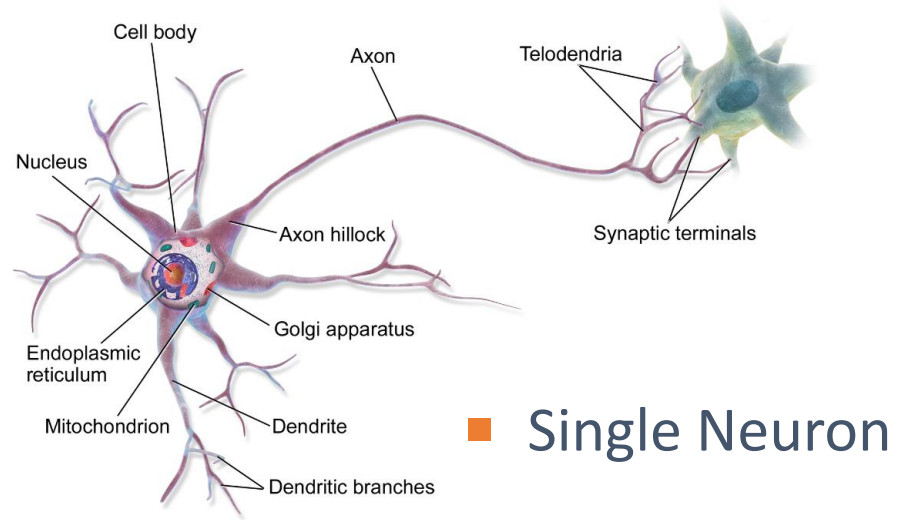


■ Single Neuron

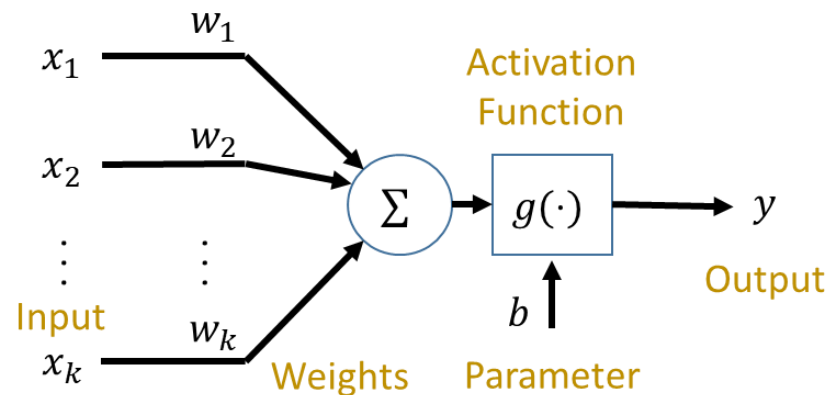




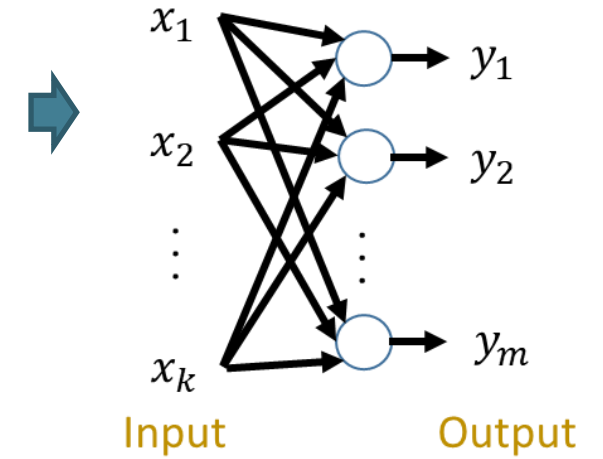
# Artificial Neural Network (NN)



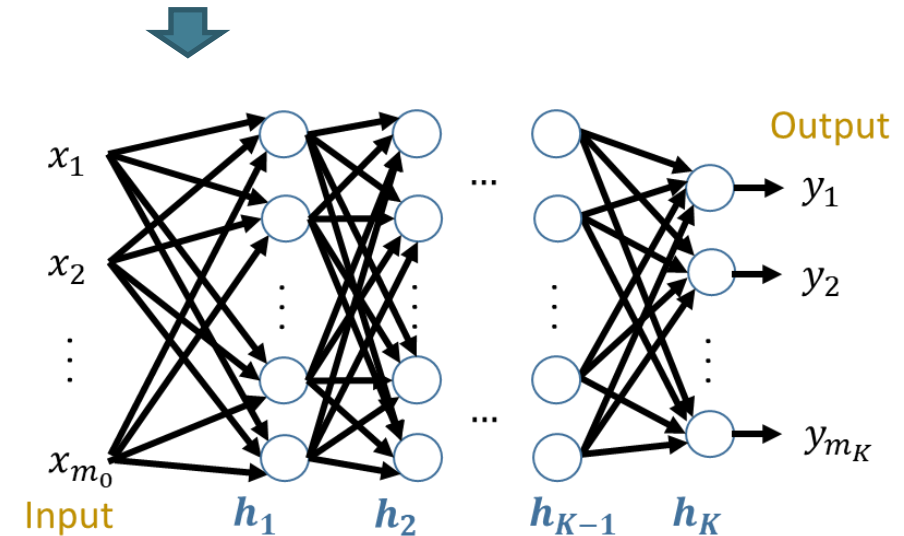
## Single Neuron



## 1-layer NN

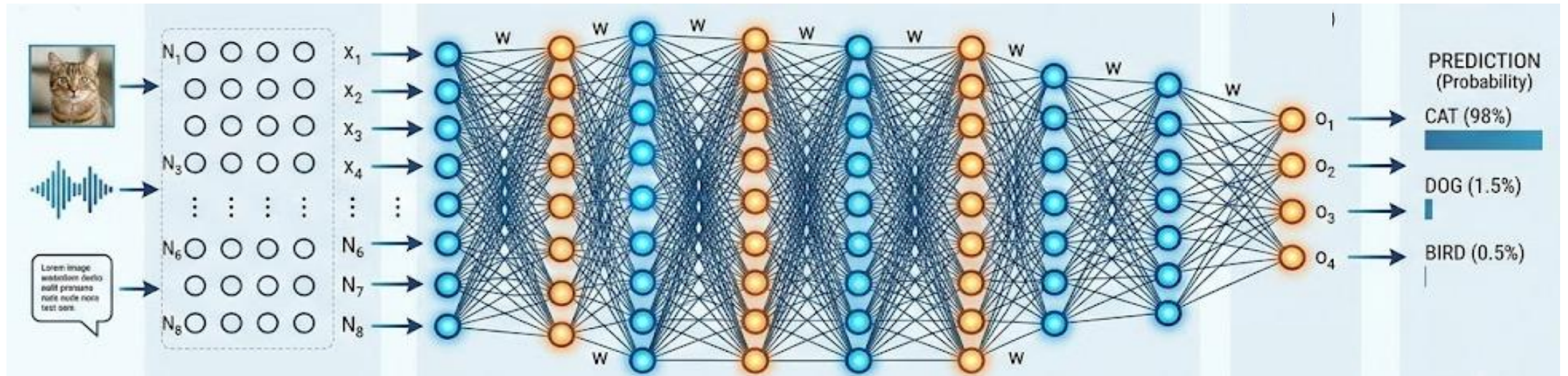


## K-layer NN



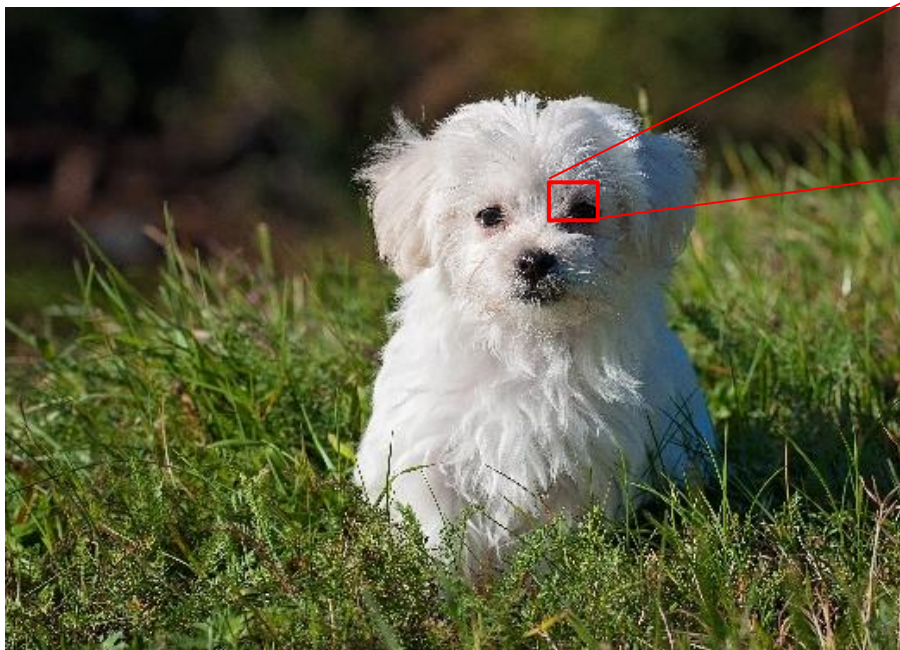
# Deep Neural Network (DNN)

- A **DNN** is a NN with multiple layers and capable of learning features from raw data
- Traditional ML requires hand-craft features (e.g., computing statistical summaries); deep learning learns features directly from raw data, reducing that manual step
- Same core math as the 1980s backprop era — what changed is data availability, GPU compute, and architectural refinements (e.g., 2017 transformer architecture)



# Understanding the Input and Output

Images are arrays of pixels value. All that the computer “sees” is a list of those numbers.



The labels are often encoded using a list as well. For example, if our list contains cats, dogs and birds, we could use a one-hot encoding:

$Cat \rightarrow [1 \ 0 \ 0]$  ;  $Dog \rightarrow [0 \ 1 \ 0]$  ;  $Bird \rightarrow [0 \ 0 \ 1]$



# How Computers Learn

- Consider the training data
    - $[1 \ 1] \rightarrow 2$
    - $[2 \ 1] \rightarrow 3$
    - $[2 \ 2] \rightarrow 4$
  - What is the output for an input  $[2 \ 1]$ ?
  - What is the output for an input  $[3 \ 2]$ ?
- Consider the training
    - $[1 \ 1] \rightarrow 3$
    - $[2 \ 1] \rightarrow 5$
    - $[2 \ 2] \rightarrow 6$
  - What is the output for an input  $[3 \ 2]$ ?
    - Relationship 1:  $[x \ y] \rightarrow 2x + y = 8$
    - Relationship 2:  $[x \ y] \rightarrow x^2 - x + y + 2 = 10$

We test an ML model by giving it data that it has not seen to ensure that it is capturing the underlying relationship. **[Generalization]**

The model needs to be properly specified given the amount of data provided. **[Overfitting / Underfitting]**

# From Tensors to Agents

**Edgar Lobaton, Ph.D.**

Director of Applied AI, College of Engineering (COE), NC State  
Professor, Electrical and Computer (ECE) Engineering, NC State  
NAIRR AI Education Fellow, Computing Research Association (CRA)

# Roadmap

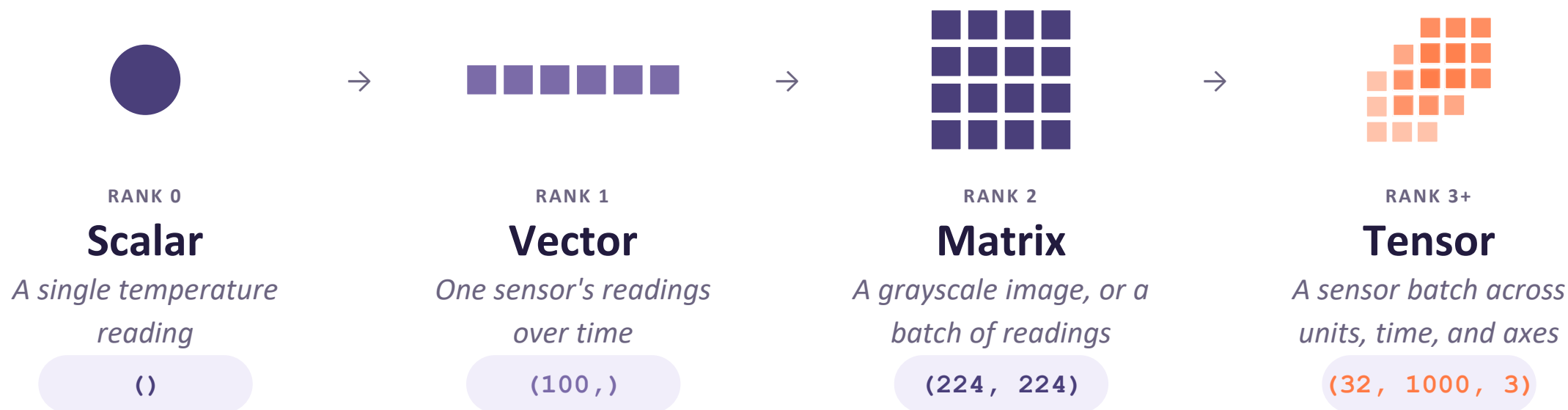
---

- We open the black box behind  
*predictive (insight) → generative (content) → agentic (action)*
- Predictive and Generative workflows  
*Text / Images → Tensors → Models → Prediction*  
*Text → Tensors → Models → Text / Image Generation*
- Getting to an Agentic Pipeline  
*Generation → Reasoning → Actions*



# What is a Tensor?

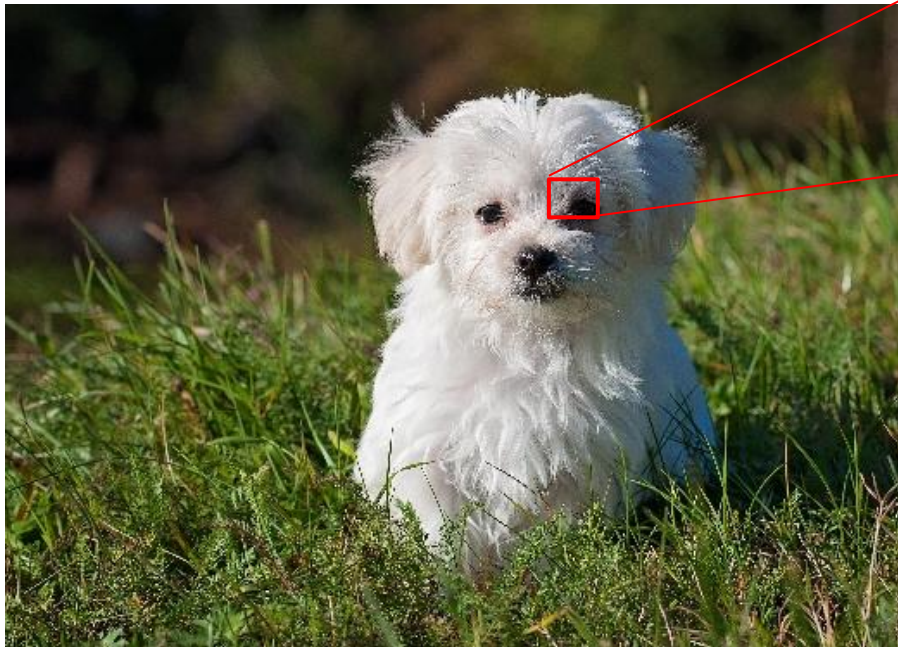
- A multi-dimensional (n-dimensional) array of numbers. Think of the generalization of a vector and a matrix.



- Every step in an AI/ML workflow uses tensor operations under the hood.

# From Images to Tensors

Images are arrays of pixels value. All that the computer “sees” is a list of those numbers.



Shape = (8,12,3)

Predictions could be class predictions:

$Cat \rightarrow [1 \ 0 \ 0]$  ;  $Dog \rightarrow [0 \ 1 \ 0]$  ;  $Bird \rightarrow [0 \ 0 \ 1]$

Object detections:

$Dog \rightarrow [x_0, y_0, x_1, y_1, c]$

Image Segmentation

# Content Generation



# From Text to Tensors



## Tokenization

Text is broken into subword units (tokens), each mapped to an integer ID — not quite words, not quite characters.

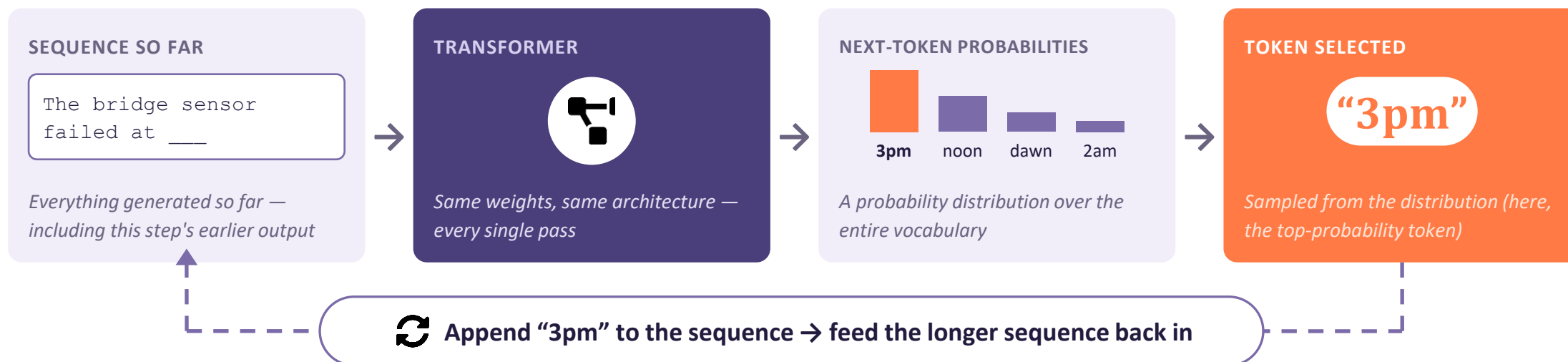
## Embeddings

Each token ID is mapped to a vector in high-dimensional space, positioned so related meanings land near each other.

- Sequences of vectors are then used as input to models that can predict class labels, sequence labeling, etc.

# Generating Text

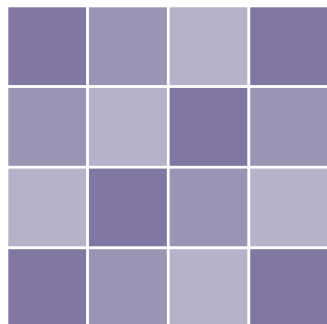
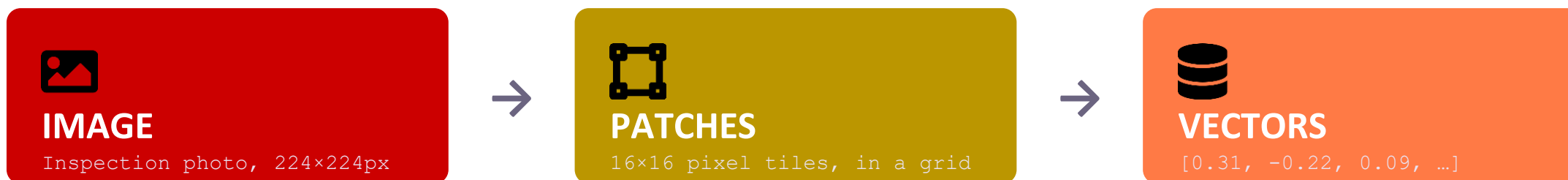
- Each new token is predicted, appended and fed back in.



This loop repeats once per generated token — a 50-token reply means running it 50 times, with the sequence growing.

- The transformer itself is stateless — all memory lives in the growing sequence fed back in real time. The **context-window** is the input to the transformer. It has to be bounded.

# From Images to Tensors (Revisited)



*Image divided into a grid of patches*

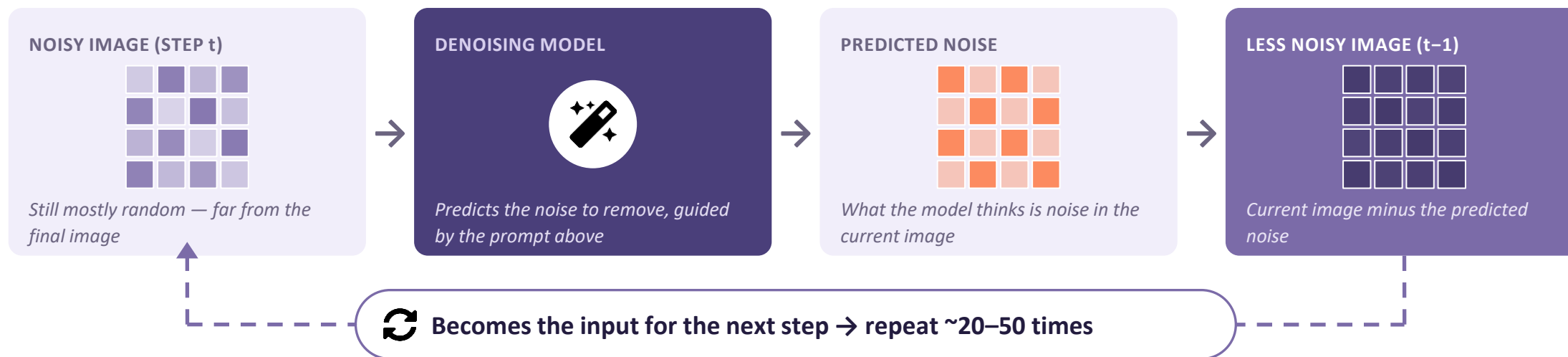
Each patch (or, in older CNN approaches, each learned filter response) becomes a vector — the same embedding-space idea from text, just applied to pixels instead of tokens.

- Input images don't need to be processed sequentially over space.

# Generating Images

- Each pass removes a little noise and feeds the results back in

**A** TEXT PROMPT: “a reinforced concrete bridge at sunset” — the same prompt conditions every single step below



This loop repeats once per denoising step — a typical image takes 20–50 steps.

- The denoising model is stateless – all memory lives in the image tensor passed between steps.



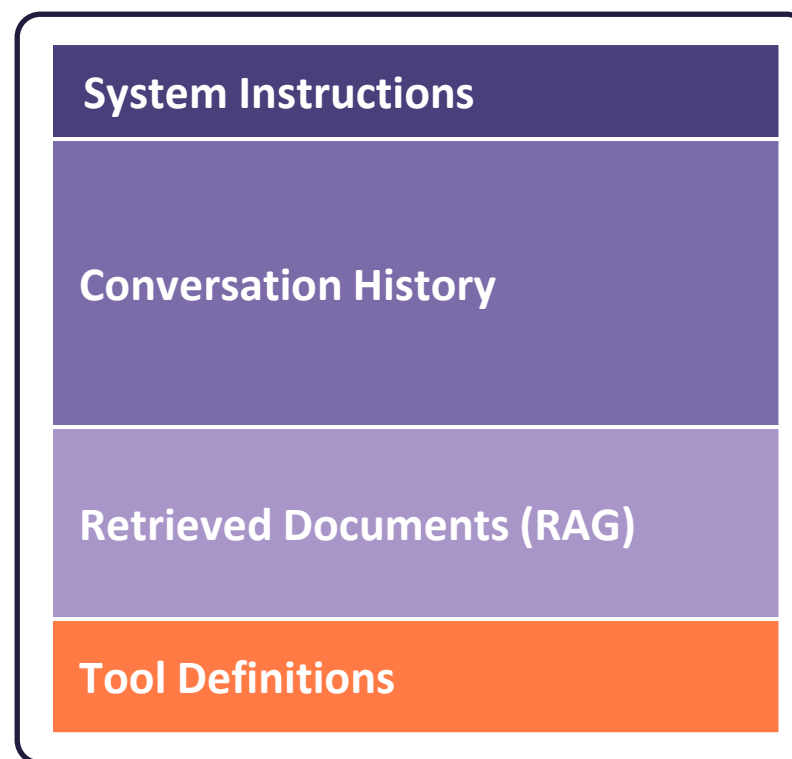
# **From Generative Model to Assistant**

# Context: The Model's Working Memory

---

- Everything the model “knows” right now lives in this one limited space
- It has no persistent memory between calls unless something explicitly puts information back into context
- Context Engineering decides what goes into this limited space, and in what order
- Once something falls outside the window, the model has no mechanism to say: “I don’t have that”. It just continues generating plausible content.

## CONTEXT WINDOW



# Personas: Shaping Behavior, Not Knowledge

---

- The same question, the same model, two very different answers



## “Patient Tutor”

*Q: Why isn't my gradient descent converging?*

Great question — let's work through it. First, check your learning rate: if it's too high, updates can overshoot the minimum and bounce around instead of settling. Try reducing it by 10x and see what happens...



## “Terse Code Reviewer”

*Q: Why isn't my gradient descent converging?*

Learning rate too high. Reduce by 10x. Also check for unnormalized features — likely culprit given your loss curve.

# Hallucinations: Confident, Not Necessarily Correct

---

- The model predicts plausible text. It has no built-in “I don’t know” signal.



## Model's apparent confidence

Consistently high — every answer sounds equally certain

## Actual factual accuracy

Varies — depends on training data and topic

- The model is producing the statistically plausible next token, not consulting a fact database. When it doesn't “know” something, it just continues the pattern rather than stopping.



# RAG vs. Fine-Tuning

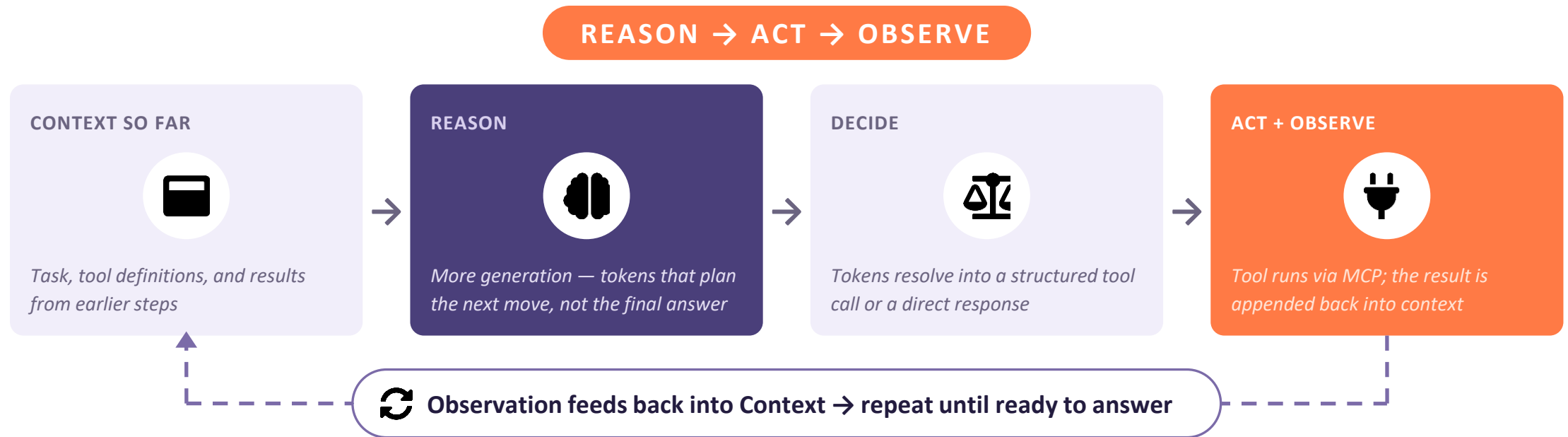
---

- RAG (Retrieval-Augmented Generation) is really an automated way of deciding what goes into Context — based on relevance, not fixed in advance.

|                    | RAG                                    | Fine-Tuning                        |
|--------------------|----------------------------------------|------------------------------------|
| What changes       | Context, at request time               | The model's weights                |
| Update speed       | Instant — swap the source              | Requires retraining                |
| Source attribution | Yes — traceable to a retrieved doc     | No — baked into weights            |
| Best for           | Fast-changing or large knowledge bases | Stable shifts in behavior or style |

# **From Generation to Action**

# Reasoning

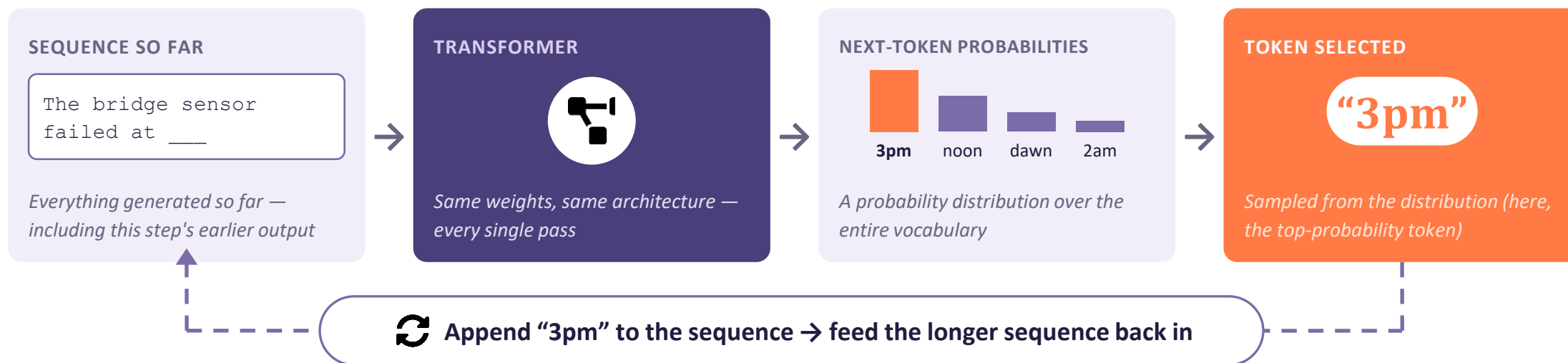


This loop can run many times — each pass may call a different tool, refine the plan, or decide it's ready to answer.

- Does this look familiar?

# Generating Text

- Each new token is predicted, appended and fed back in.



This loop repeats once per generated token — a 50-token reply means running it 50 times, with the sequence growing.

- The transformer itself is stateless — all memory lives in the growing sequence fed back in real time. The **context-window** is the input to the transformer. It has to be bounded.

# Skills: Packaged Procedural Expertise

---

- Reusable knowledge the model reads before doing a specific class of tasks



**Instructions, examples, templates — read before the task, not baked into weights**

**Engineering analogy:** a skill is functionally a standard operating procedure (SOP). A specialist doesn't re-derive the correct inspection procedure from first principles each time — they follow a written procedure refined over many iterations.

*This is now a first-class concept in production agent platforms — not just an informal prompting trick.*

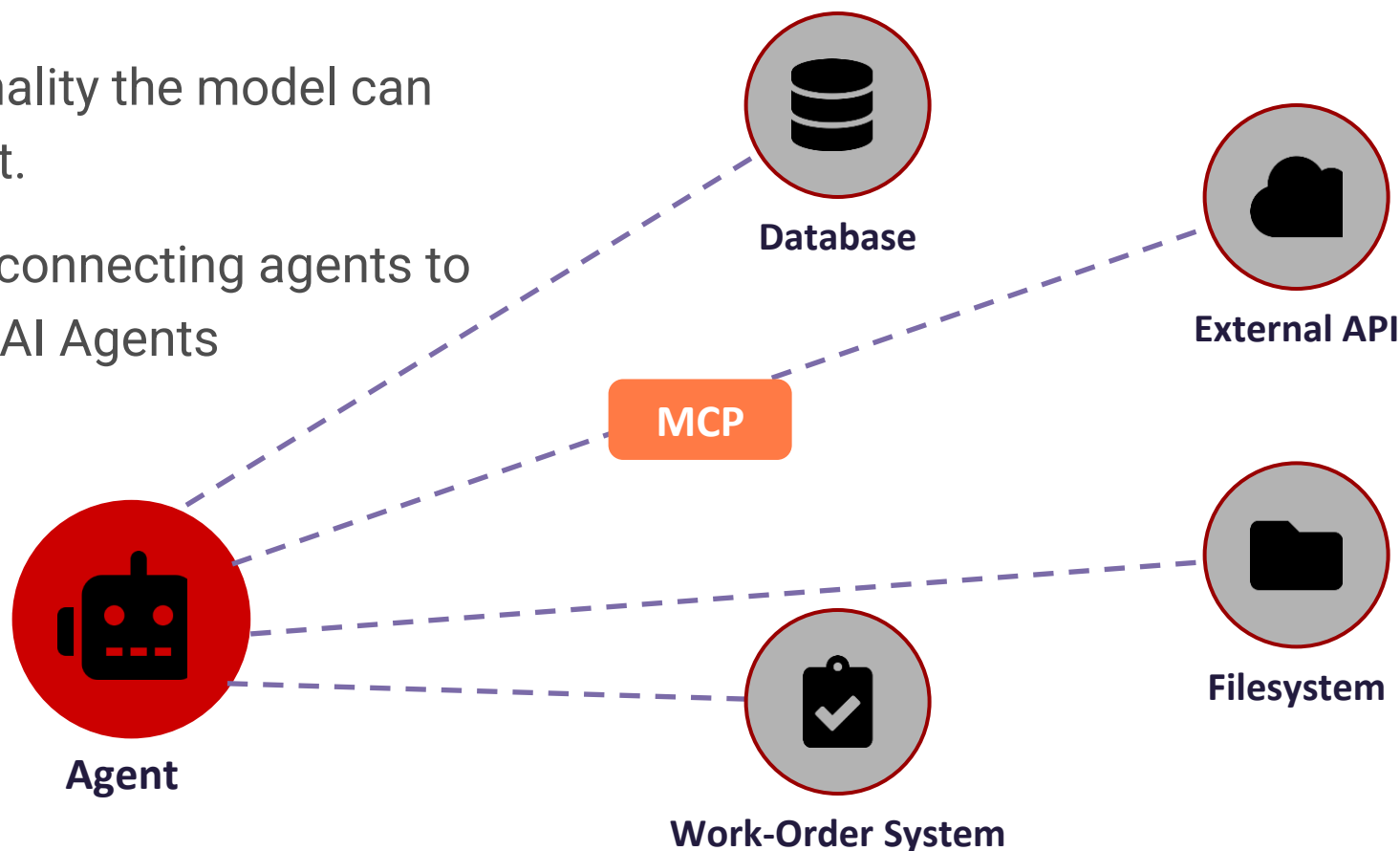
- While Personas set the role, Skills supply the know-how for a specific tasks within that role



# MCP: Model Context Protocol

---

- A tool is any external functionality the model can invoke beyond generating text.
- MCP is an open standard for connecting agents to tools and data, “the USB-C of AI Agents”



# Putting It All Together

- An agent incorporates a persona, with skills, calling tools via MCP, grounded by RAG, making repeated generative calls in a loop.

